```java
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;


public class JavaStreamAPIExam {

    static class Employee {
        final String name;
        final String department;
        final double salary;

        Employee(String name, String department, double
salary) {
            this.name = name;
            this.department = department;
            this.salary = salary;
        }
    }

    public static void main(String[] args) {
        question1();
        question2();
        question3();
        question4();
        question5();
        question6();
        question7();
        question8();
        question9();
        question10();
    }

    private static void question1() {
        String input = "swiss";
        Character result = input.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(
                Function.identity(), LinkedHashMap::new,
Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() == 1)
            .map(Map.Entry::getKey)
            .findFirst()
```

```
                .orElse(null);

        System.out.println("Q1: " + result);
    }

    private static void question2() {
        List<Integer> numbers = Arrays.asList(10, 20, 30,
20, 40, 10, 50, 30);
        List<Integer> duplicates = numbers.stream()
                .collect(Collectors.groupingBy(Function.ident
ity(), Collectors.counting()))
                .entrySet().stream()
                .filter(entry -> entry.getValue() > 1)
                .map(Map.Entry::getKey)
                .sorted()
                .collect(Collectors.toList());

        System.out.println("Q2: " + duplicates);
    }

    private static void question3() {
        List<Integer> numbers = Arrays.asList(10, 5, 20,
20, 8, 15, 10);
        Integer result = numbers.stream()
                .distinct()
                .sorted(Comparator.reverseOrder())
                .skip(1)
                .findFirst()
                .orElse(null);

        System.out.println("Q3: " + result);
    }

    private static void question4() {
        String sentence = "Java Spring Java Hibernate
Spring Java";
        Map<String, Long> frequency =
Arrays.stream(sentence.split("\\s+"))
                .map(String::toLowerCase)
                .collect(Collectors.groupingBy(
                    Function.identity(), LinkedHashMap::new,
Collectors.counting()));

        System.out.println("Q4:");
```

```
        frequency.forEach((word, count) ->
System.out.println(word + "=" + count));
    }

    private static void question5() {
        List<String> words = Arrays.asList(
            "Java", "SpringBoot", "Microservices",
"Hibernate", "Docker");
        String result = words.stream()
            .sorted(Comparator.comparingInt(String::lengt
h).reversed())
            .findFirst()
            .orElse(null);

        System.out.println("Q5: " + result);
    }

    private static void question6() {
        List<Employee> employees = Arrays.asList(
            new Employee("Ahmed", "IT", 10000),
            new Employee("Mohamed", "IT", 15000),
            new Employee("Sara", "HR", 8000),
            new Employee("Omar", "HR", 12000),
            new Employee("Ali", "Finance", 20000));

        Map<String, Double> averages = employees.stream()
            .collect(Collectors.groupingBy(
                e -> e.department, LinkedHashMap::new,
                Collectors.averagingDouble(e ->
e.salary)));

        System.out.println("Q6:");
        averages.forEach((department, average) ->
            System.out.println(department + " = " +
average));
    }

    private static void question7() {
        List<Employee> employees = Arrays.asList(
            new Employee("Ahmed", "IT", 10000),
            new Employee("Mohamed", "IT", 18000),
            new Employee("Sara", "HR", 9000),
            new Employee("Omar", "HR", 14000),
            new Employee("Ali", "Finance", 20000));
```

```
        Map<String, Employee> highestPaid =
employees.stream()
                .collect(Collectors.groupingBy(
                        e -> e.department, LinkedHashMap::new,
                        Collectors.collectingAndThen(
                                Collectors.maxBy(Comparator.comparingDouble(e ->
e.salary)),
                                Optional::get)));

        System.out.println("Q7:");
        highestPaid.forEach((department, employee) ->
                System.out.println(department + " = " +
employee.name));
    }

    private static void question8() {
        List<Integer> list1 = Arrays.asList(1, 2, 3, 4,
5, 6, 3);
        List<Integer> list2 = Arrays.asList(4, 5, 6, 7,
8, 4);

        List<Integer> common = list1.stream()
                .filter(list2::contains)
                .distinct()
                .collect(Collectors.toList());

        System.out.println("Q8: " + common);
    }

    private static void question9() {
        List<Integer> numbers = Arrays.asList(
                1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 6);

        Map<Integer, Long> frequency = numbers.stream()
                .collect(Collectors.groupingBy(Function.ident
ity(), Collectors.counting()));

        System.out.println("Q9:");
        frequency.entrySet().stream()
                .sorted(Comparator.<Map.Entry<Integer, Long>,
Long>
                        comparing(Map.Entry::getValue).reversed()
```

```
                .thenComparing(Map.Entry::getKey))
            .limit(3)
            .forEach(entry -> System.out.println(
                entry.getKey() + " = " + entry.getValue()
+ " occurrences"));
    }


    private static void question10() {
        List<Employee> employees = Arrays.asList(
            new Employee("Ahmed", "IT", 10000),
            new Employee("Mohamed", "IT", 18000),
            new Employee("Ali", "IT", 15000),
            new Employee("Omar", "IT", 22000),
            new Employee("Sara", "HR", 12000),
            new Employee("Mona", "HR", 16000),
            new Employee("Nour", "HR", 14000),
            new Employee("Hana", "HR", 9000),
            new Employee("Khaled", "Finance", 20000),
            new Employee("Youssef", "Finance", 25000),
            new Employee("Mostafa", "Finance", 18000));

        Map<String, List<Employee>> topEmployees =
employees.stream()
            .collect(Collectors.groupingBy(
                e -> e.department, LinkedHashMap::new,
                Collectors.collectingAndThen(
                    Collectors.toList(), list ->
list.stream()
                        .sorted(Comparator.comparingDoubl
e((Employee e) -> e.salary)
                            .reversed().thenComparing(e
-> e.name))
                        .limit(3)
                        .collect(Collectors.toList()))));

        System.out.println("Q10:");
        topEmployees.forEach((department, list) -> {
            System.out.println(department + ":");
            list.forEach(e -> System.out.println(e.name +
" " + (int) e.salary));
        });
    }
}
```